

IMPLEMENTATION TIMELINE & DELIVERY PLAN

YESE

OpenTek School Enterprise Management System (SEMS-ERP)

A 12-Week, Phase-Based Implementation Plan — September to December 2026

Version 2.0 — Aligned to the Master Implementation Prompt & SRS v1.1

Prepared by KWIDJA Marc & NOKA Gildas — OpenTek

Date: 29 September 2026

Table of Contents

1. Overview & Approach

This plan schedules the implementation of YESE — OpenTek's multi-tenant, modular School Enterprise Management System (SEMS-ERP) — across a 12-week period from September to December 2026. Unlike the earlier draft, this version is aligned directly to the architecture and phase order defined in the project's Master Implementation Prompt and SRS: a Modular Monolith with strong module boundaries, built phase by phase (0 → 1 → 1.5 → 2 → ...).

Guiding principle: YESE is not a collection of CRUD pages. It is a multi-tenant modular platform where independent business modules are assembled and configured per institution over a secure common core. The schedule below is built to progressively prove that architecture, not merely to ship features.

1.1 Planning parameters

Parameter	Value
Platform name	YESE
Duration	12 weeks (September – December 2026)
Core team	Two developers (KWIDJA Marc & NOKA Gildas), supported by the wider OpenTek team
Backend	NestJS + TypeScript, REST APIs, OAuth 2.0
Frontend	Next.js + TypeScript + Redux-Toolkit + Tailwind
Database	PostgreSQL — hosted online (managed cloud DB) for direct shared access
Architecture	Modular Monolith with strong module boundaries
Method	Agile / Scrum, phase-based, incremental (test as we go)

1.2 Realistic scope statement

A full enterprise ERP — academics plus finance, HR, payroll, communication, portals and the Phase 7 extensions — is a large, multi-month undertaking for a full team. A core team of two cannot build the entire ERP in 12 weeks, and this plan does not pretend otherwise.

What the 12 weeks deliver: the complete platform foundation (Phase 0–1), the module-activation engine (Phase 1.5), and the core educational lifecycle (Phase 2–3) — exactly the MVP defined in Section 24 of the Master Prompt: multi-tenancy, authentication, RBAC, module activation, student, staff, academic structure, enrollment, attendance, assessment, results and basic reporting. Finance (Phase 4) and Communication/Portals (Phase 5) are scheduled as stretch goals, and everything in Phase 7 is deferred.

Why this is the right MVP: it demonstrates at least two different tenant configurations (e.g. Tenant A = Core+Student+Academic+Attendance; Tenant B = the same plus Assessment+Results), which proves module activation is real and not theoretical — the single most important thing to show OpenTek.

2. Phases

The 12 weeks follow the phase order mandated by the Master Implementation Prompt. Testing runs continuously (Rule 7), concentrating at the end.

Phase	Weeks	Focus & exit criterion
Phase 0 — Preparation	W1	Runnable dev environment: monorepo, NestJS + Next.js, hosted PostgreSQL, env config, Git workflow, conventions.
Phase 1 — Platform Foundation	W1–4	Tenant management, identity & authentication, users, two-layer RBAC, configuration, audit logging. Exit: create tenant → create user → authenticate → resolve tenant → assign role → check permission → access protected resource.
Phase 1.5 — Modular Architecture	W4–5	Module Registry: metadata, dependencies, per-tenant activation/deactivation, module permissions. Exit: the platform can answer 'is this module active for this tenant, are its dependencies met, does this user have permission?'
Phase 2 — School Core	W5–8	Student, Staff, Academic structure, Enrollment. Exit: create academic year → class/section → staff → student → enroll → assign teacher.
Phase 3 — Academic Operations	W8–11	Attendance, Assessment, Results, Report cards, basic reporting. Exit: a full academic lifecycle from enrollment to report card.
Phase 4 — Finance (stretch)	W10–12	Fee structures, invoices, payments, receipts, immutable financial records. Included only if Phases 0–3 finish on schedule.
Phase 5 — Communication & Portals (stretch)	W11–12	Notifications and role portals, respecting the communication firewall. Stretch.
Phase 6 — Testing & Stabilization	continuous → W12	Unit/integration/e2e, tenant-isolation and module-activation tests, security review, demo prep.
Phase 7 — Advanced Extensions	post-MVP	Library, transport, payroll, POS, integrations, analytics, AI — deferred.

3. Phase Delivery Schedule

The chart maps each phase to the 12 weeks (in fortnights). Darker cells indicate primary build effort; lighter cells indicate start-up, continuation or wind-down. Testing is continuous, so it carries light cells across most of the schedule.

Phase / Workstream	W1–2	W3–4	W5–6	W7–8	W9–10	W11–12
Phase 0 — Environment & Setup						
Phase 1 — Platform Foundation (tenancy, auth, RBAC, audit)						
Phase 1.5 — Module Registry (activation engine)						
Phase 2 — School Core (Student, Staff, Academic, Enrollment)						
Phase 3 — Academic Ops (Attendance, Assessment, Results)						
Phase 4 — Finance (stretch)						
Phase 5 — Communication & Portals (stretch)						
Phase 6 — Testing & Stabilization (continuous)						
Integration & Demo preparation						

Legend: ■ primary build ■ start-up / continuation

4. Milestones & Deadlines

Each phase has a target completion (deadline) at end of the stated week.

Milestone	Deadline	Definition of done
M0 — Environment ready	End W1	Repo runs; backend, frontend and hosted PostgreSQL connected; env configured.
M1 — Foundation live	End W4	Tenancy, auth, users, RBAC, config and audit working end-to-end; protected resource reachable.
M1.5 — Modularity proven	End W5	A module can be activated/deactivated per tenant with dependency + permission checks.
M2 — School core	End W8	Academic structure, staff, students and enrollment operational.
M3 — Academic lifecycle	End W11	Attendance → assessment → results → report card works; basic reporting.
M4 — Finance (stretch)	End W12	Invoice → payment → receipt with immutable records (if reached).
M6 — MVP demo	End W12	Two tenant configurations demonstrated; tested, stabilized, release-candidate.

5. Post-MVP Roadmap (deferred)

Deferred to increments after the 12-week MVP, added one module at a time on the proven foundation and sold per package tier:

Increment	Modules
Next	Finish Finance; Communication, Notifications, Portals; HR & Leave
Then	Payroll; Timetable/Scheduling; Reporting & Dashboards depth
Later (Phase 7)	Library, Transport, Hostel, Cafeteria, Inventory, POS, Procurement, Online Payments, SMS/Email/WhatsApp, LMS, Analytics, AI, Alumni, Document Management

6. Risks, Assumptions & Ways of Working

6.1 Key risks

Phase 1.5 complexity: the Module Registry (per-tenant activation with dependency resolution) is the hardest early piece; a small team can lose a week here. It is scheduled with its own phase for that reason.

Small core team: two developers limit parallel work; wider-team support is upside, not assumed in the estimates.

Scope pressure: the ERP breadth invites scope creep; the MVP boundary (through Phase 3) must be protected, with Finance/Comms as explicit stretch.

Two-layer RBAC & tenant isolation: subtle to get right; must be enforced at the data-access layer, not the frontend. Build early, test explicitly.

6.2 Assumptions to confirm

1. The database is PostgreSQL, hosted online (managed cloud) for direct shared team access. (This resolves the earlier Postgres-vs-Mongo ambiguity in favour of PostgreSQL.)
2. The MVP target is Phases 0–3; Finance and Communication/Portals are stretch within the 12 weeks.
3. The two of us are the accountable core; wider OpenTek support accelerates but is not required to meet the plan.
4. The stack (NestJS / Next.js / PostgreSQL) is fixed, so ramp-up is short rather than a full new-language learning curve.

6.3 Ways of working

Work incrementally: explain → show structure → implement → run → test → verify architecture → commit (Master Prompt Section 26).

Each module independently deliverable and demoable; integrate continuously to avoid a big-bang merge.

Weekly check-ins tracking phase progress against the milestones in Section 4.

— End of Implementation Timeline & Delivery Plan (YESE), Version 2.0 —